

Analysis: Food Stalls

Test set 1 (Visible)

We will start by fixing the position of the warehouse. Once that is done, we know the cost of putting up a stall at any spot. This allows for a dynamic programming approach where we let $dp[i][j]$ represent the cost of putting j stalls in the first i spots. $dp[i][j]$ is minimum of $dp[i-1][j]$ and $(dp[i-1][j-1] + \text{distance of } i\text{th spot from warehouse} + C_i)$ (we skip i if we have chosen to place the warehouse at i). $dp[N][K] + \text{cost of building the warehouse}$ represents our solution for the chosen location of the warehouse.

By doing this for all possible positions for the warehouse, we will get the minimum cost for placing all K stalls. So this approach has a complexity of $O(N^2K)$.

Test set 2 (Hidden)

Now suppose we decided on the $K+1$ spots where we will place our stalls and warehouse but are yet to decide where to place our warehouse, let's say the co-ordinates of these spots are $Y_1, Y_2, \dots, Y_{K+1}$ in increasing order. Obviously, it'll be the spot Y_j such that $|Y_j - Y_1| + |Y_j - Y_2| + \dots + |Y_j - Y_{K+1}|$ is minimum. This is the classic post office location problem and is solved by putting the warehouse in the median point of our chosen points (in case of even number of points with two medians, any one will yield an optimal answer).

Once we have this observation, we know that we must put $\text{floor}(K/2)$ stalls on the left of our warehouse and $K - \text{floor}(K/2)$ stalls on the right. If we can calculate, for every position of the warehouse, the minimum cost to place these points, we will be done. Here we will show how to calculate the minimum cost to place $\text{floor}(K/2)$ stalls to the left of every position of the warehouse.

For simplicity, we assume the given points $X_1, X_2, \dots, X_N$ to be sorted. Now, we will maintain a max-heap of size $\text{floor}(K/2)$. For position i , we define $V_i = X_N - X_i + C_i$. Initially, we will store $V_1, V_2, \dots, V_{\text{floor}(K/2)}$ in our heap. We'll also maintain the sum of all the elements in our heap.

If we consider that we are placing the warehouse at $X_{\text{floor}(K/2)+1}$, then the minimum cost of placing all $\text{floor}(K/2)$ stalls to the left of it is given by sum of all the elements in our heap - $\text{floor}(K/2) * (X_N - X_{\text{floor}(K/2)+1})$. We try to get this cost for all other valid placements of the warehouse.

So, for $i = \text{floor}(K/2) + 1$ to $N - (K - \text{floor}(K/2)) - 1$, we check if V_i is less than the maximum element of our heap. If it is then we remove the maximum and insert V_i and update the sum of all our heap elements by subtracting the difference of the removed element and the inserted element. We subtract $\text{floor}(K/2) * (X_N - X_{i+1})$ from the sum and compare it with the minimum cost obtained thus far, updating it with the current cost if current cost is lower.

We can use a similar approach to get the cost of all elements to the right, there V_i will just be X_i and we should iterate from right to left and in each iteration, subtract $(K - \text{floor}(K/2)) * X_i$ from the sum of the elements in the heap and update the minimum cost with that.

Finally we see for which point i , the sum of cost of placing $\text{floor}(K/2)$ stalls on the left of it, $K - \text{floor}(K/2)$ stalls on the right and C_i is minimised and output this value.

Analysis: Latest Guests

Test set 1 (Visible)

For this test set, we simply simulate the entire affair. For each second, we update the positions of all the guests and will add them to the list of latest guests list of the consulate that they visit. We also keep track of the minute when the list for each consulate was updated so that if a guest comes at a later time, we discard the old list and put this guest (and possibly others) to a new list. Finally after the M th minute, we iterate through all these lists and find for each guest, how many lists they were a part of and output the answer. This takes a total of $O(G \times (M+N))$ time which is sufficient for this test set.

Test set 2 (Hidden)

Let's simplify the problem by assuming all the guests travel clockwise. We can get the final position of each guest after M minutes by simply taking their current position, adding M to it and finding the modulo of the result with N . We categorise the guests into groups based upon their final positions. All guests having the same final positions are in the same group and guests having different final positions are in different groups. We maintain a mapping from guests to groups. We are going to find for each group, how many consulates remember it.

Now notice that for any consulate only the group whose final position is on or just after it in a clockwise order can be the last visiting group. We can get this by doing a binary search on the sorted list of final positions of the groups or using a sliding window. Note that there can be at most one group which will be remembered by a consulate.

If this group did not visit this consulate then no one has visited it. To find this we find the time when the group would have visited this consulate last. So we take the difference of their final positions and the position of the consulate and subtract this number from M . If the resultant is negative that means no one visited this consulate. Otherwise, for this consulate, we make a note of this group and the time when they visited.

We handle anti-clockwise guests similarly and get for each consulate the last anti-clockwise guest group to visit it and the time when the group visited.

Now we iterate through the consulates and see which was the last group to visit it (we include both the clockwise and the anti-clockwise groups). We then increment the number of consulates that remembers this group. Finally, we iterate through the guests and see which group they belong to and output the number of consulates that remember this group. This solution has a run-time of $O(N+G)$ if implemented using the sliding window technique or a run-time of $O(G + N \log(N))$ if we use binary-search instead.

Analysis: X or What?

Test set 1 (Visible)

Let's define a new array S . We set $S_0 = 0$, $S_1 = A_1$ and $S_i = S_{i-1} \text{ xor } A_i$ for $i = 2$ to N (Note that S is zero-indexed while A is one-indexed). We can see that once we've calculated this, $A_1 \text{ xor } A_2 \text{ xor } \dots \text{ xor } A_r$ is simply given by $S_r \text{ xor } S_{i-1}$.

With this, Test set 1 can be solved just by calculating the xor sum of every sub-interval of A and checking if it's *xor-even*. After each update, we need to recompute S which only takes $O(N)$ time. So each query can be handled in $O(N^2)$ time with an overall complexity of $O(QN^2)$.

Test set 2 (Hidden)

Let's extend the definition of *xor-even* to mean any number having even number of 1s in its binary representation, similarly for *xor-odd*. Now, notice that if we xor two *xor-even* numbers or two *xor-odd* numbers (numbers having an odd number of 1s in their binary representations), we get a *xor-even* number and, similarly, if we xor a *xor-even* number with a *xor-odd* number, we get a *xor-odd* number. Hence, if there are an even number of *xor-odd* numbers in an interval then that interval is going to be *xor-even* and vice versa.

This means that if there are even number of *xor-odd* numbers in our array, the whole array is *xor-even*. Otherwise, we consider the subarray starting just after the first *xor-odd* number and going till the end and the subarray starting from the first element in our array and ending just before the last *xor-odd* number. Both are *xor-even* intervals and the larger of them should be the largest *xor-even* interval in our array.

We can do this by keeping a set of all positions of *xor-odd* numbers. Every time we update a number, we simply do an insertion or a deletion or leave the set unchanged. If the size of the set is even, then the whole array is *xor-even*, otherwise we get the left most and the right most positions from this set and output the answer as discussed above.

Since we will have an $O(N)$ elements in the set and there will be Q queries, each of $O(\log(N))$ time, this solution has a complexity of $O((N+Q)\log(N))$.

We can also solve this problem using [van Emde Boas trees](#) in $O(Q\log(\log(N)))$. Or we can use offline algorithms as well to achieve even better asymptotic solutions. Figuring out the details of these approaches is left as exercises to the reader.